

重慶理工大學

实践项目报告书

实践项目：数据库设计

实践题目：个人课程表管理系统数据库设计

班　　级：

学　　号：

姓　　名：

联系电话：

指导老师：

实验时间：2026.10—2026.11

同组同学：

计算机科学与工程学院　计算机实验室（中心）

目　录

1　项目背景　2

1.1　选题背景 2

1.2　系统目标 2

2　系统需求分析　3

2.1　业务数据流程分析 4

2.2　数据字典 7

3　概念结构设计　9

4　逻辑结构设计　11

4.1　关系模式与联系转换 12

4.2　关系规范化分析 12

5　物理结构设计　13

5.1　基本表 SQL 脚本 13

5.2　数据库生成结果与查询验证 15

6　总结　18

7　参考文献　18

1 项目背景

1.1 选题背景

高校学生的日常课程安排具有较强的阶段性与灵活性。一个学期内，同一门课程往往拆分为理论教学与上机实验，分别安排在不同星期、节次和教室；部分课程采用单双周交替或阶段性周次（如第 1～8 周、第 9～16 周）授课，选修课、重修课或辅修课之间还可能出现局部周次的时间重叠。

在日常学习生活中，许多同学仍习惯借助纸质课表、手机截图或单张电子表格整理课表，管理上存在三方面不便：

（1）数据冗余且修改繁琐：同一门课程在每周多次上课时，需要反复填写课程名称与授课信息，一旦教室或时间调整，极易出现漏改和前后不一致；

（2）单双周与离散周次筛选不便：静态表格难以按“当前第几周”动态过滤当周生效的课程，遇到隔周实验或临时停课调课时仍需人工逐条核对；

（3）选课冲突难以及时发现：增补选修课或跨专业课程时，仅凭肉眼比对星期、节次与周次区间，容易遗漏仅在特定周次发生的排课重叠。

针对上述痛点，设计并建立结构化的个人课程表管理数据库十分必要。通过将课表配置、课程信息、具体排课与生效周次分层存储，既能消除重复录入、保证数据一致性，又能便捷地支持按周课表展示与时间冲突提醒。

1.2 系统目标

本系统面向学生个人日常排课与查课需求，主要实现以下六项功能目标：

（1）课表配置：支持按学期创建与管理多张独立课表，自定义每张课表的第一周起始日期、学期起止周次以及每日各节次的起止时间；

（2）课程维护：在指定课表内对课程信息进行增删改查，支持同一门课程配置多条不同时段、不同地点的上课安排；

（3）周次管理：支持全周、单周、双周及任意不连续周次的录入与展开存储，满足实验课、短期课等灵活排课需求；

（4）课程查询：支持按课表、教学周次、星期以及课程名称、教师或地点关键字检索课程，并关联输出具体上下课时间；

（5）导入导出：支持通过课表交换文件批量导入或导出整张课表数据，便于跨学期复用或备份迁移；

（6）冲突提示：自动识别同一课表内在相同星期、重叠节次且存在共同上课周次的课程安排，向用户发出冲突提示并支持按需保留。

系统功能结构如图 1-1 所示。

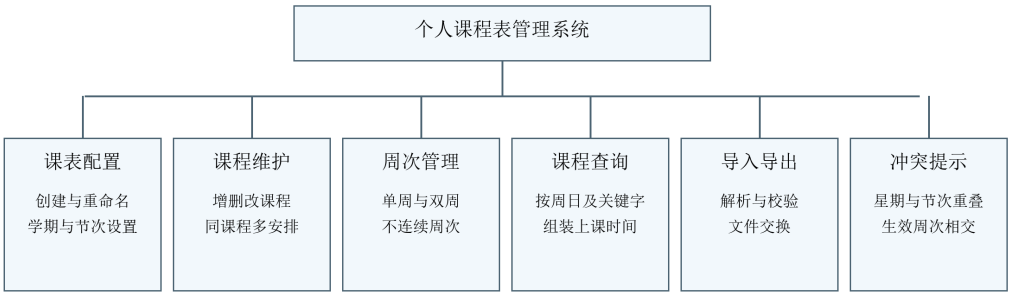


图 1-1　系统功能目标图

在技术方案上，系统采用 B/S 三层体系结构，表现层负责课表视图展示与表单交互，业务逻辑层负责课表文件解析、单双周集合展开及事务控制，数据层负责持久化存储。数据库管理系统选用 MySQL 8.0（InnoDB 存储引擎，`utf8mb4` 字符集），建模阶段使用 Sybase PowerDesigner 15.1 绘制业务处理模型（BPM）、概念数据模型（CDM）与物理数据模型（PDM），并借助 Docker 容器环境与 Adminer 管理工具完成本地建库及 SQL 查询测试。

2　系统需求分析

围绕个人日常排课与查课场景，按照信息要求、处理要求以及安全性与完整性要求三个维度梳理系统需求[1,2]：

（1）信息要求：系统需以“课表”为单位隔离不同学期或不同方案的数据，记录五类核心信息：一是课表基本信息（名称、第一周起始日期、起止周次、创建与修改时间）；二是作息节次定义（节次序号及每日起止时间）；三是课程基本信息（课表内课程编号、名称与备注）；四是上课安排（星期、起止节次、授课教师、教室地点及备注）；五是每次上课安排对应的具体生效周次及单周说明。

（2）处理要求：支持课表及节次作息的初始化配置与修改；支持单门课程录入、多排课时段维护以及交换文件的批量导入与导出；支持按指定周次、星期或关键字快速生成周课表视图；支持检测同一课表内的排课重叠并反馈冲突详情。考虑到重修免听或隔周选课等现实情况，检测到时间冲突时仅向用户预警，经用户确认后仍允许保存。

（3）安全性与完整性要求：数据库服务仅监听本地回环地址，并为业务访问分配限定于本库权限的专用账号，结合定期逻辑备份保障数据安全。完整性方面，除利用主键与复合外键确保课程、节次、排课及周次严格归属于同一张课表外，还需校验第一周起始日为星期一、星期取值 1～7、节次时间有序不重叠、生

效周次落在学期起止周范围内，并通过事务机制保证批量导入与整表删除的原子性。

2.1 业务数据流程分析

系统总体业务数据流程围绕课表配置（P1）、课程导入与维护（P2）、课程查询（P3）和文件导出（P4）四项核心处理展开，外部实体为个人用户（E1，图中为避免流向交叉在四条支路分别绘制了同义复本），内部包含 D1～D5 五个数据存储，如图 2-1 所示。



图 2-1　系统总体数据流程图

2.1.1 课表配置业务流程

用户新建或修改课表时，提交课表名称、第一周起始日期、起止周次及每日节次作息表。系统首先在 P1.1 中校验各项时间与周次参数是否合法，若存在错误则直接反馈提示；校验通过后由 P1.2 在同一事务内写入 D1 课表与 D2 节次时间，并返回配置结果（图 2-2）。

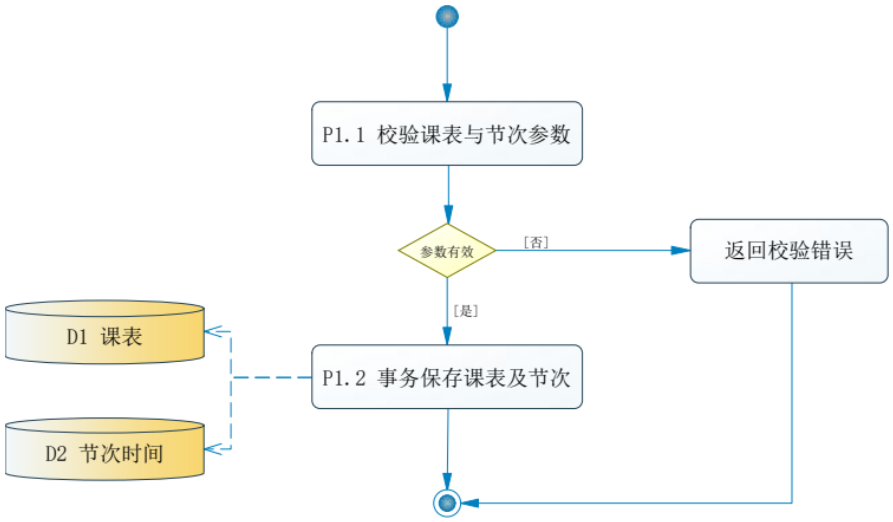


图 2-2　课表配置 BPM 图

2.1.2　课程导入与维护业务流程

用户手工录入课程或导入课表文件时，P2.1 先解析并校验课程、节次与周次字段；随后 P2.2 将单双周、连续周等规则展开为具体周次集合，并与已有排课比对时间冲突。用户确认保存后，P2.3 在事务内同步更新 D3 课程、D4 上课安排与 D5 上课周次，若写入异常则整体回滚（图 2-3）。

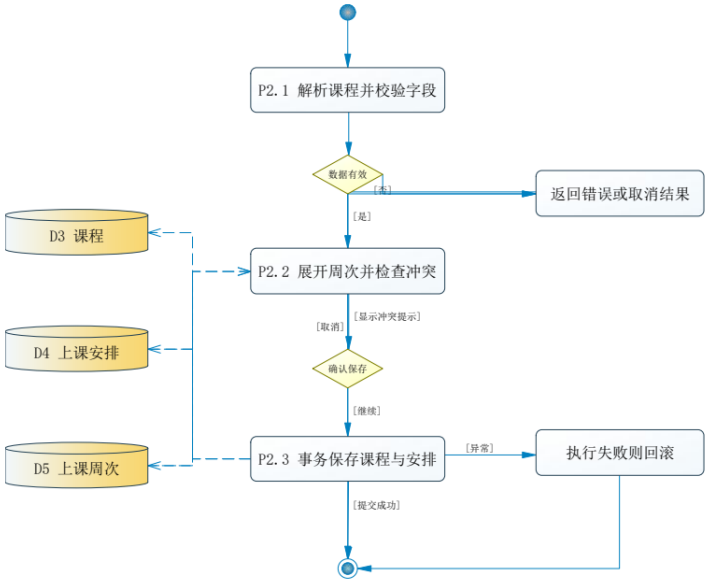


图 2-3　课程导入与维护 BPM 图

2.1.3　课程查询与导出业务流程

查询或导出课表时，P3.1 先检查目标课表编号与筛选周次条件，再由 P3.2 关联读取 D1～D5 中的课程、排课、节次时间与生效周次，按星期和节次顺序组装课表结果。若用户发起导出请求，则继续进入 P4.1 将整张课表结构化输出为课表交换文件（图 2-4）。

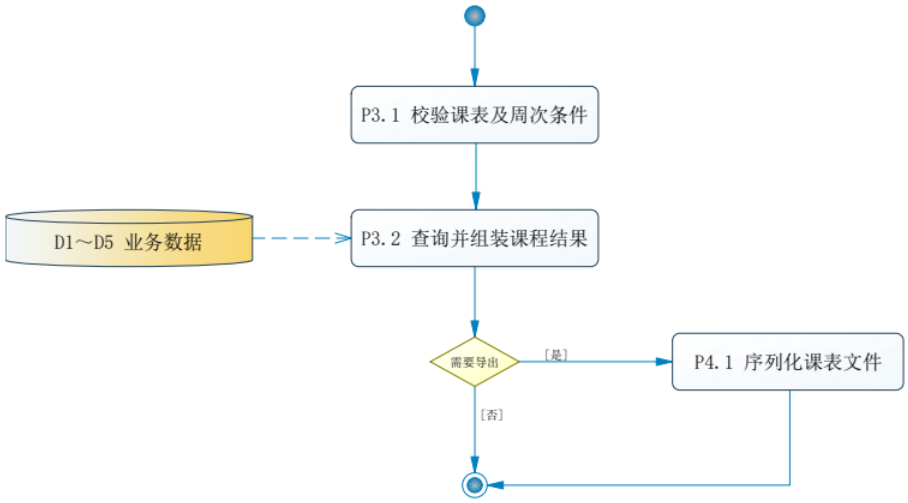


图 2-4　查询与导出 BPM 图

2.2 数据字典

依据上述分层数据流程图，按数据流、数据存储、数据处理、外部实体和数据项五部分编写系统数据字典如下。

2.2.1 数据流

编号	名称	来源 → 去向	组成及说明	估计流量
F1	配置请求	E1 → P1	课表编号、名称、第一周起始日期、起止周及节次时间集合	2 ~ 5 次/学期
F2	配置结果	P1 → E1	操作状态、课表编号、更新时间或参数校验提示	2 ~ 5 次/学期
F3	课程数据	E1 → P2	目标课表编号、课程信息、上课安排集合及生效周次集合	10 ~ 30 次/学期
F4	维护结果	P2 → E1	保存状态、课程与安排编号、时间冲突提示或错误原因	10 ~ 30 次/学期
F5	查询条件	E1 → P3	课表编号、周次、星期及课程/教师/地点筛选关键字	5 ~ 20 次/天
F6	查询结果	P3 → E1	课程名称、星期、起止节次及时间、教师、地点与备注列表	5 ~ 20 次/天
F7	导出请求	E1 → P4	目标课表编号与导出文件格式参数	1 ~ 3 次/学期
F8	课表文件	P4 → E1	包含课表配置、节次、课程、排课与周次的完整交换文件	1 ~ 3 次/学期

2.2.2 数据存储

编号	存储名称	记录组成	组织方式与用途
D1	课表 `timetable`	课表编号、课表名称、第一周起始日期、起始周、结束周、创建时间、修改时间	按课表编号索引组织；存储各学期课表基本参数
D2	节次时间 `period_time`	课表编号、节次编号、开始时间、结束时间	按（课表编号，节次编号）组织；存储每日作息时段
D3	课程 `course`	课表编号、课程编号、课程名称、课程备注	按（课表编号，课程编号）组织；存储课表内课程信息
D4	上课安排 `course_meeting`	课表编号、课程编号、安排编号、星期、开始节次、结束节次、教师姓名、上课地点、安排备注	按（课表编号，课程编号，安排编号）组织；存储每周排课时段与教室
D5	上课周次 `meeting_week`	课表编号、课程编号、安排编号、生效周次、周次备注	按四元联合键组织；按周记录排课生效情况

2.2.3 数据处理

编号	处理名称	输入、处理逻辑与输出	处理频率
P1	课表配置	接收 F1；经 P1.1 校验起始日、周次范围与节次顺序后，由 P1.2 事务写入 D1、D2，输出 F2	学期初或作息调整时
P2	课程导入与维护	接收 F3；经 P2.1 校验字段、P2.2 展开单双周并比对 D4/D5 检测冲突后，由 P2.3 事务更新 D3～D5，输出 F4	选课阶段或调课时
P3	课程查询	接收 F5；经 P3.1 校验课表与周次条件，由 P3.2 连接 D1～D5 按星期和节次排序组装结果，输出 F6	日常高频按需查询
P4	文件导出	接收 F7；读取指定课表下 D1～D5 完整关联记录，由 P4.1 序列化生成交换文件 F8	学期末备份或迁移时

2.2.4 外部实体

编号	名称	说明及相关数据流
E1	个人用户	课表所有者，负责发起配置、排课维护、按周查询与导出请求（输入 F1、F3、F5、F7，接收 F2、F4、F6、F8）

2.2.5 数据项

编号	数据项与字段代码	类型	取值范围与说明
I01	课表编号 `timetable_id`	int	正整数，课表唯一标识，非空
I02	课表名称 `timetable_name`	varchar(50)	1～50 字符，非空
I03	第一周起始日期 `term_start_date`	date	学期第一周星期一对应日期，非空
I04	起始周 `start_week`	smallint	1～53，且不大于结束周，默认 1
I05	结束周 `end_week`	smallint	1～53，且不小于起始周，默认 20
I06	创建时间 `created_at`	datetime	记录创建时间，非空
I07	修改时间 `updated_at`	datetime	最近修改时间，不早于创建时间，非空
I08	节次编号 `period_no`	smallint	1～24，同一课表内唯一，非空
I09	开始时间 `start_time`	time	每日节次开始时刻，早于结束时间，非空
I10	结束时间 `end_time`	time	每日节次结束时刻，晚于开始时间，非空
I11	课程编号 `course_no`	int	正整数，同一课表内唯一，非空
I12	课程名称 `course_name`	varchar(100)	1～100 字符，非空
I13	课程备注 `course_remark`	varchar(200)	课程补充说明，可为空
I14	安排编号 `meeting_no`	int	正整数，同一课程内唯一，非空
I15	星期 `day_of_week`	smallint	1～7（对应星期一至星期日），非空
I16	开始节次 `start_period`	smallint	引用所属课表节次，不大于结束节次，非空
I17	结束节次 `end_period`	smallint	引用所属课表节次，不小于开始节次，非空
I18	教师姓名 `teacher_name`	varchar(100)	授课教师姓名，可为空
I19	上课地点 `location`	varchar(100)	教室或场地名称，可为空
I20	安排备注 `meeting_remark`	varchar(200)	理论/实验等排课备注，可为空
I21	生效周次 `week_no`	smallint	1～53，位于所属课表起止周内，非空
I22	周次备注 `week_note`	varchar(200)	单周次调课或特殊说明，可为空

3 概念结构设计

在需求分析基础上，从排课业务中抽象出“课表”、“节次时间”、“课程”、“上课安排”与“上课周次”五个实体，借助 PowerDesigner 构建系统概念数据模型（CDM）[3]，

如图 3-1 所示（图中标注 <pi> 的属性构成主标识，标注 <M> 的属性为强制非空项）。

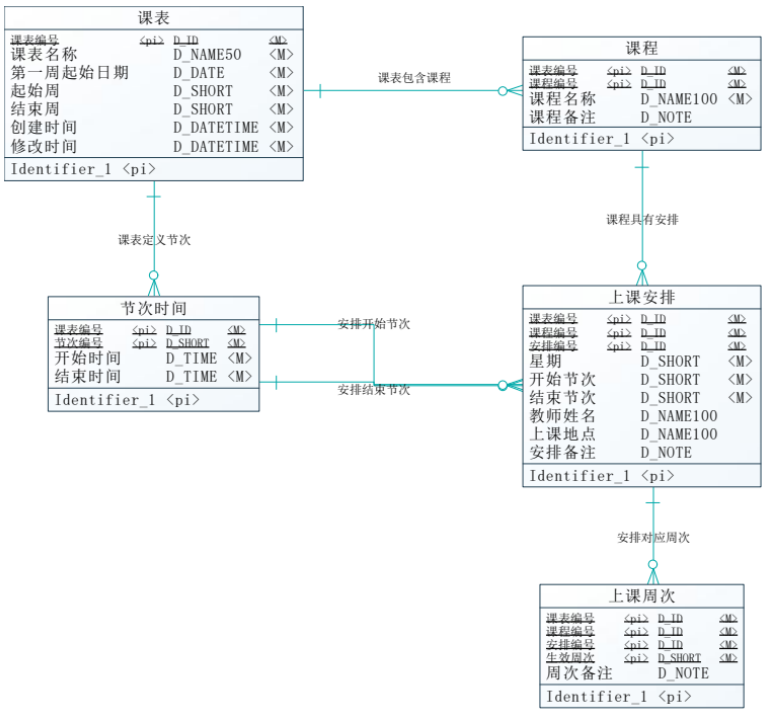


图 3-1　个人课程表管理系统 CDM 图

考虑到不同学期的课表相互独立，且同一学期内可能出现重名选修课或重名教师，名称类属性难以充当稳定标识。因此模型采用层次化联合主标识：以“课表编号”作为顶层标识，下属的“课程”叠加“课程编号”，“上课安排”叠加“安排编号”，“上课周次”进一步叠加“生效周次”；“节次时间”则由“课表编号 + 节次编号”共同标识。这种从属标识设计直观体现了课表内部编号的唯一性，同时便于在物理阶段直接构建同课表约束。

为保证同类属性定义规范统一，CDM 预设了八个可复用数据域：D_ID（整型编号）、D_SHORT（周次、节次与星期短整型）、D_NAME50 与 D_NAME100（名称文本）、D_NOTE（备注文本），以及 D_DATE、D_TIME、D_DATETIME 三种日期时间域。实体间共建立六条一对多（1:N）联系，具体基数与业务含义汇总如下：

联系名称（实体间）	联系类型	业务含义与依赖说明
课表包含课程（课表—课程）	1:N	一张课表可包含多门课程，每门课程仅归属一张课表
课表定义节次（课表—节次时间）	1:N	一张课表定义多个日内节次时段，各课表作息互不干扰
课程具有安排（课程—上课安排）	1:N	同一门课程可设置理论课、实验课等多个每周固定时段
安排开始/结束节次（节次—安排）	1:N（两条）	上课安排的起止节次分别引用所属课表内的节次定义
安排对应周次（安排—上课周次）	1:N	一次上课安排对应若干具体生效周次，支持单双周与离散周

4　逻辑结构设计

将 CDM 转换为面向 MySQL 8.0 的物理数据模型（PDM），并整理自动迁移生成的复合外键列，合并同一课表上下文中的公共标识字段。转换后的系统 PDM 如图 4-1 所示。

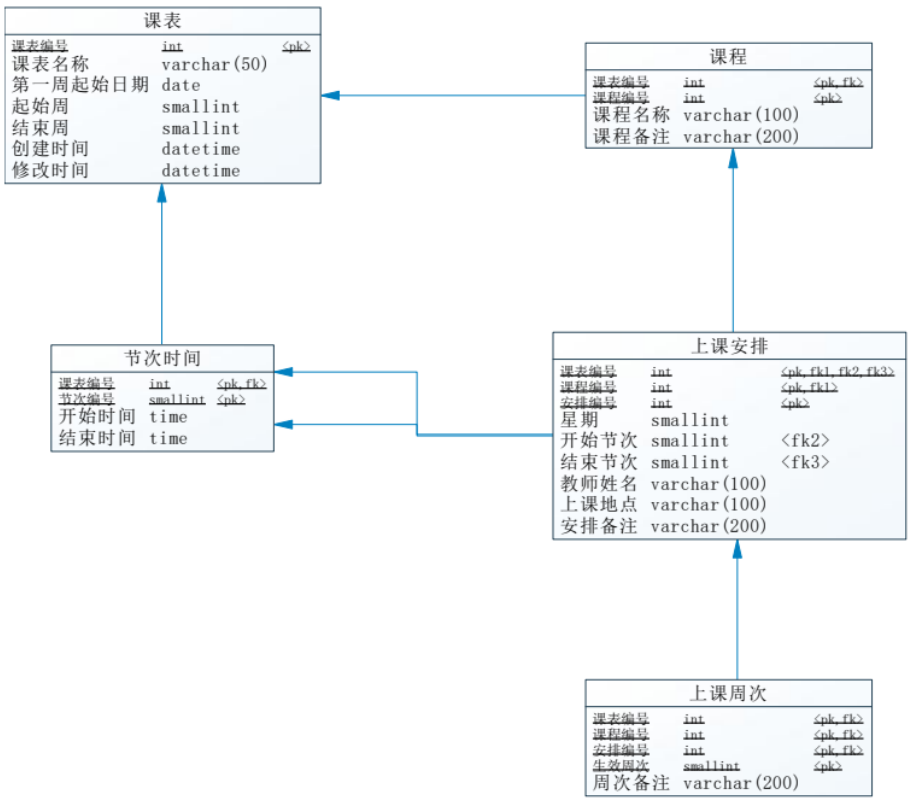


图 4-1　个人课程表管理系统 PDM 图

4.1 关系模式与联系转换

由于概念模型中各实体之间均为一对多（1:N）联系，按 E-R 图向关系模型的转换规则[1]，均可将“一”端实体的主码并入“多”端关系作为外键，无需额外拆分多余的独立联系表；对于表达多值生效周次的 meeting_week 表，除四元联合主码外还设计了非主属性 week_note，用于记录单周调课或停课说明。转换得到的五个关系模式如下（PK 表示主键，FK 表示外键）：

（1）课表 timetable（timetable_id，timetable_name，term_start_date，start_week，end_week，created_at，updated_at）

PK = （timetable_id）。

（2）节次时间 period_time（timetable_id，period_no，start_time，end_time）

PK =（timetable_id，period_no）；FK：timetable_id 引用 timetable（timetable_id）。

（3）课程 course（timetable_id，course_no，course_name，course_remark）

PK =（timetable_id，course_no）；FK：timetable_id 引用 timetable（timetable_id）。

（4）上课安排 course_meeting（timetable_id，course_no，meeting_no，day_of_week，start_period，end_period，teacher_name，location，meeting_remark）

PK =（timetable_id，course_no，meeting_no）；FK：（timetable_id，course_no）引用 course，（timetable_id，start_period）与（timetable_id，end_period）分别引用 period_time（timetable_id，period_no）。

（5）上课周次 meeting_week（timetable_id，course_no，meeting_no，week_no，week_note）

PK =（timetable_id，course_no，meeting_no，week_no）；FK：（timetable_id，course_no，meeting_no）引用 course_meeting。

4.2 关系规范化分析

按关系规范化理论逐级检验上述五张关系表[1,2]：

（1）第一范式（1NF）：所有属性均为不可再分的原子值。课程的多时段排课拆分为 course_meeting 的多行记录，生效周次也在 meeting_week 中按单周一行存放，避免了用逗号拼接周次字符串带来的查询与维护困难。

（2）第二范式（2NF）：在采用复合主码的四张表中，由于不同课表之间的课程编号、节次编号相互独立，课程名称完全函数依赖于（timetable_id，course_no），排

课的星期、起止节次、教师与地点完全依赖于(timetable_id,course_no,meeting_no)，周次备注也完全依赖于四元联合主码，消除了非主属性对码的部分函数依赖。

（3）第三范式（3NF）：课程名称仅保存在 course 表中，具体上下课时刻仅保存在 period_time 表中，学期周次范围仅保存在 timetable 表中，各表非主属性之间相互独立，不存在对主码的传递函数依赖，因此五个关系模式均达到 3NF。

此外，针对高频查询场景，在 PDM 中补充了四个辅助索引：ix_tt_updated（updated_at）用于按更新时间排列课表，ix_c_name（timetable_id，course_name）支持课表内按课程名称检索，ix_cm_day（timetable_id，day_of_week，start_period，end_period）用于按星期排课排序与冲突筛查，ix_mw_week（timetable_id，week_no，course_no，meeting_no）用于加速按教学周提取当周课表。

5 物理结构设计

5.1 基本表 SQL 脚本

在 PowerDesigner 中将目标 DBMS 指定为 MySQL 8.0[4]，导出建库 DDL 脚本并在目标库 personal_timetable 中执行。为节省篇幅，以下保留由 PDM 生成的脚本头文件及五张基本表的建表核心语句（外键约束、辅助索引及补充跨表周次/节次校验的触发器与整表删除存储过程 sp_delete_timetable 已随数据库脚本一并部署）：

```
/*==============================================================*/
/* DBMS name:      MySQL 8.0                                    */
/* Created on:     2026/11/7 19:52:12                           */
/*==============================================================*/
```

课表表

```
create table timetable
(
   timetable_id int not null,
   timetable_name varchar(50) not null,
   term_start_date date not null,
   start_week smallint not null default 1,
   end_week smallint not null default 20,
   created_at datetime not null default CURRENT_TIMESTAMP,
   updated_at datetime not null default CURRENT_TIMESTAMP,
   primary key (timetable_id),
   check ((timetable_id > 0)
      AND (CHAR_LENGTH(TRIM(timetable_name)) BETWEEN 1 AND 50)
      AND (start_week >= 1
      AND end_week <= 53
      AND start_week <= end_week)
      AND (WEEKDAY(term_start_date) = 0)
      AND (created_at <= updated_at))
)
ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

节次时间表

```
create table period_time
(
   timetable_id int not null,
   period_no smallint not null,
   start_time time not null,
   end_time time not null,
   primary key (timetable_id, period_no),
   check ((period_no BETWEEN 1 AND 24)
      AND (start_time >= '00:00:00'
      AND end_time <= '23:59:59'
      AND start_time < end_time))
)
ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

课程表

```
create table course
(
   timetable_id int not null,
   course_no int not null,
   course_name varchar(100) not null,
   course_remark varchar(200),
   primary key (timetable_id, course_no),
   check ((course_no > 0)
      AND (CHAR_LENGTH(TRIM(course_name)) BETWEEN 1 AND 100))
)
ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

上课安排表

```
create table course_meeting
(
   timetable_id int not null,
   course_no int not null,
   meeting_no int not null,
   day_of_week smallint not null,
   start_period smallint not null,
   end_period smallint not null,
   teacher_name varchar(100),
   location varchar(100),
   meeting_remark varchar(200),
   primary key (timetable_id, course_no, meeting_no),
   check ((meeting_no > 0)
      AND (day_of_week BETWEEN 1 AND 7)
      AND (start_period <= end_period))
)
ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

上课周次表

```
create table meeting_week
(
   timetable_id int not null,
   course_no int not null,
   meeting_no int not null,
   week_no smallint not null,
   week_note varchar(200),
   primary key (timetable_id, course_no, meeting_no, week_no),
   check ((week_no BETWEEN 1 AND 53))
)
ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

5.2 数据库生成结果与查询验证

执行建表与完整性脚本后，在 MySQL 8.0 中生成的数据库对象如图 5-1 所示，左侧目录树与右侧表清单中已正确建立 course、course_meeting、meeting_week、period_time、 timetable 五张基本表及课表删除存储过程。



图 5-1　数据库基本表生成结果截图

为检验表结构对核心业务的支撑效果，向库中导入“2026 秋季学期”与“2027 春季学期”两套示例课表（涵盖理论实验分时排课、单双周、离散周次及重修冲突样例）。查询第 1 张课表第 3 周生效课程时，通过 meeting_week 连接课程表、排课表与起止节次时间即可生成按星期和节次排列的周课表：

```
SELECT c.course_name, m.day_of_week, m.start_period, m.end_period,
       p1.start_time, p2.end_time, m.teacher_name, m.location
FROM meeting_week w
JOIN course_meeting m USING (timetable_id, course_no, meeting_no)
JOIN course c USING (timetable_id, course_no)
JOIN period_time p1 ON p1.timetable_id = m.timetable_id AND p1.period_no = m.start_period
JOIN period_time p2 ON p2.timetable_id = m.timetable_id AND p2.period_no = m.end_period
WHERE w.timetable_id = 1 AND w.week_no = 3
ORDER BY m.day_of_week, m.start_period, c.course_no;
```

针对同课表内的排课冲突检测，可通过自连接 course_meeting 与 meeting_week 筛选相同星期、节次区间重叠且存在共同生效周次的课程组合，并通过主键偏序条件避免同一组冲突正反重复输出：

```sql
SELECT a.timetable_id, a.course_no AS course_a, a.meeting_no AS meeting_a,
       b.course_no AS course_b, b.meeting_no AS meeting_b, a.day_of_week,
       GROUP_CONCAT(wa.week_no ORDER BY wa.week_no) AS common_weeks
FROM course_meeting a
JOIN course_meeting b
  ON a.timetable_id = b.timetable_id
 AND (a.course_no < b.course_no
      OR (a.course_no = b.course_no AND a.meeting_no < b.meeting_no))
 AND a.day_of_week = b.day_of_week
 AND a.start_period <= b.end_period AND b.start_period <= a.end_period
JOIN meeting_week wa
  ON wa.timetable_id = a.timetable_id
 AND wa.course_no = a.course_no AND wa.meeting_no = a.meeting_no
JOIN meeting_week wb
  ON wb.timetable_id = b.timetable_id
 AND wb.course_no = b.course_no AND wb.meeting_no = b.meeting_no
 AND wb.week_no = wa.week_no
GROUP BY a.timetable_id, a.course_no, a.meeting_no,
         b.course_no, b.meeting_no, a.day_of_week
ORDER BY a.timetable_id, a.course_no, b.course_no;
```



图 5-2　课程时间冲突查询结果截图

如图 5-2 所示，查询准确识别出第 3 周星期一的两组节次重叠记录（第 1～2 节“数据库原理及应用”、第 3～4 节“大学英语”分别与第 2～3 节“程序设计实践”发生交叉），而星期、节次相同但上课周次错开的课程不会误报。测试完成后，已使用 `mysqldump` 导出完整结构与示例数据备份文件留存。

6 总结

本次数据库课程设计围绕“个人课程表管理系统”，完整实践了从需求分析、BPM 业务数据流建模、数据字典编写，到 CDM 概念设计、PDM 逻辑转换与关系规范化，再到 MySQL 8.0 建库测试的全流程。在建模过程中，体会最深的是合理拆分实体粒度与设计复合主外键的重要性：将“课程”、“上课安排”与“上课周次”拆分为三层一对多关系后，不仅自然兼容了同一课程每周多次授课、单双周及离散周次等复杂情况，而且无需借助字符串拆分即可用标准连接语句完成按周查课与冲突检测；在子表中沿用 `timetable_id` 构建复合外键，也从数据库层面避免了不同学期课表之间的错误引用。同时，通过将 PowerDesigner 生成的脚本放到 MySQL 中调试，我对 `CHECK` 约束、外键删除限制以及触发器处理跨表规则的边界有了更直观的认识。

受限于课程设计周期与个人使用定位，当前设计仍存在一些不足与可改进之处：

（1）尚未配套实现可视化的前端课表界面与教务系统课表文件解析程序，目前主要通过 SQL 脚本验证核心存取与查询逻辑；

（2）授课教师与上课地点仅作为文本字段记录在排课表中，若后续扩展为支持班级共享或校园公共空闲教室查询的系统，则应将教师、教学楼与教室独立建表管理；

（3）对于单周临时换教室或补课场景，目前仅通过 `meeting_week` 表的 `week_note` 备注字段做文字记录，尚不支持覆盖原安排的节次与教室字段，后续可引入调课变动表进一步完善。

7 参考文献

[1] 王珊, 萨师煊. 数据库系统概论（第 5 版）[M]. 北京: 高等教育出版社, 2014: 199-245.

[2] 李唯唯, 尹静, 黄丽丰, 程勇军. 数据库原理及应用：微课视频版 [M]. 北京: 清华大学出版社, 2020: 1-299.

[3] Sybase Inc. PowerDesigner DataArchitect User’s Guide (Version 15.1)[Z]. 2009.

[4] Oracle Corporation. MySQL 8.0 Reference Manual[EB/OL]. https://dev.mysql.com/doc/refman/8.0/en/，访问日期：2026-11-06。